

HOOP – Human Object Orientated Programming Language

HOOP Documentación

1. Planteamiento del problema y solución adoptada

1.1 Objetivo: ¿Por qué crear HOOP? ¿Qué problemas resuelve?

HOOP se fundamenta en resolver la dificultad de aprendizaje de lenguajes sintácticamente complejos, además de resolver la ambigüedad de palabras que abundan dentro de estos lenguajes; esta falta de novedad en los términos utilizados va de la mano con la falta de legibilidad dentro de su semántica. Nuestro lenguaje se propone manejar un lenguaje entendible para humanos, con semántica clara y sintaxis tipo pseudocódigo estructurado.

2. Descripción del lenguaje

2.1 HOOP adopta una mezcla de los paradigmas:

- Imperativo, al permitir definir instrucciones secuenciales y estructuras de control explícitas.
- Orientado a objetos, al utilizar *mold* para clases, *action* para métodos, *create* para instanciación, y *self* para referencia interna
- Posee un **tipado híbrido**, permitiendo al usuario decidir cuando sea explícito.

2.2 Palabras reservadas

2.2.1 Estructuras de definición

<i>mold</i>	clase (class)
<i>bond</i>	interfaz (interface)
<i>listing</i>	enumeración (enum)
<i>layout</i>	estructura (struct)
<i>unit</i>	paquete/módulo (package)

2.2.1 Control de flujo

<i>when</i>	Condicional if
<i>otherwise</i>	Condicional else
<i>cycle</i>	Bucle for
<i>repeat</i>	Bucle while
<i>halt</i>	Terminar bucle (break)
<i>skip</i>	Saltar iteración (continue)
<i>answer</i>	Retornar de función (return)
<i>select</i>	Estructura switch/case
<i>case</i>	Caso específico en select
<i>default</i>	Caso por defecto en select

2.2.3 Declaración

fixed	Constante (const)
data	Variable con tipado genérico/inferido (var)

2.2.4 Funciones

action	Definir función o método (fun)
--------	---

2.2.5 Objetos / referencias

forge	Crear objeto (new)
self	Referencia a la instancia actual (this)
void	Valor nulo (null)

2.2.6 Manejo de errores

attemp	Bloque try
ensure	Bloque finally
throw	Lanzar excepción (throw)
rescue	Bloque catch

2.2.7 I/O

display	Imprimir en pantalla (print / printf)
---------	---

2.3 Palabras reservadas de tipos de datos (tipado híbrido)

2.3.1 Simples

logic	Booleano (boolean)
whole	Entero (integer)
fract	Decimal / flotante (float / double)
text	Cadena de texto (string)
char	Carácter (char)

2.3.2 Compuestos

grid	Matriz (matrix)
chain	Lista enlazada (linked list)

2.4 Palabras pregonadas (funciones internas/built-in)

length	Longitud de un arreglo o cadena
size	Tamaño de una colección
type	Tipo de dato de un valor
convert	Conversión de tipo
input	Entrada de datos por consola
output	Salida de datos
read	Leer archivo o recurso
write	Escribir archivo o recurso
open	Abrir recurso
close	Cerrar recurso
max	Valor máximo
min	Valor mínimo
abs	Valor absoluto
sqrt	Raíz cuadrada
pow	Potencia
random	Número aleatorio

2.5 Operadores

2.5.1 Aritméticos

plus	Suma (+)
minus	Resta (-)
times	Multiplicación (*)
divide	División (/)
mod	Módulo (%)

2.5.2 Comparación

equals	Igual (==)
notequals	Diferente (!=)
less	Menor (<)
greater	Mayor (>)
lesseq	Menor o igual (<=)
greatereq	Mayor o igual (>=)

2.5.3 Asignación

set	Asignación (=)
-----	----------------

2.5.3 Lógicos

and	Lógico Y (&&)
or	Lógico O (**)
not	Negación (!)

2.6 Símbolos de estructura y puntuación

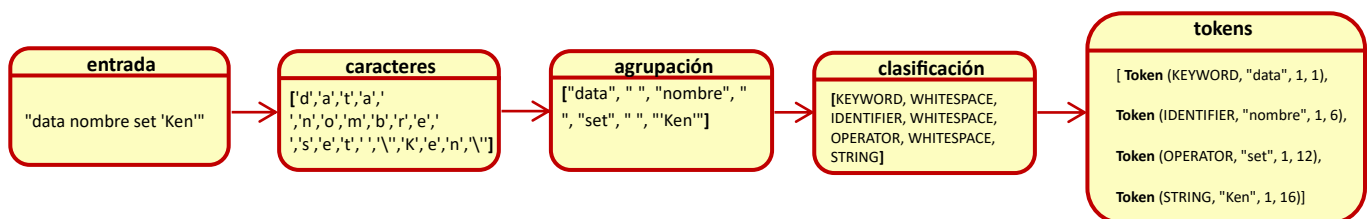
;	Fin de línea * Finaliza una instrucción
{ }	Llaves * Agrupar bloques de código
()	Paréntesis * Llamadas a funciones, agrupaciones lógicas
[]	Corchetes * Acceso a elementos en colecciones/matrices
,	Coma * Separador de elementos y de decimales
.	Punto * Acceso a miembros de un objeto
:	Dos puntos * Uso especial (por definir en el lenguaje)
+	Comentario

3. Funcionamiento de HOOP.

3.1 Etapa de análisis léxico.

Su “**LEXER**” posee un flujo de creación de Tokens, recibiendo la entrada completa, pasando por la lectura carácter por carácter, para luego agruparlos según los espacios vacíos encontrados, y clasificarlos según corresponda [KEYWORD, WHITESPACE, IDENTIFIER, WHITESPACE, OPERATOR, WHITESPACE, STRING], por último genera los tokens según corresponda con una estructura:

TOKEN (<TokenType>, <TokenContent>, <TokenColumn>, <Token Row>)



3.2 Etapa análisis sintáctico

Para HOOP se diseñó un parser estilo YACC (Yet Another Compiler Compiler). Este parser analiza la secuencia de tokens generados por el leer mencionado en el paso anterior, y los utiliza para construir un **Arbol de Sintaxis Abstracta (AST)** o detectar errores sintácticos. No usa librerías externas; todo es implementado manual. Este parser importa listas del lexer como RESERVADAS (mold, when, cycle...), TIPOS(whole, fract, logic...) OPERADORES_PALABRAS (set, plus, minus...) y PALABRAS_PREGONADAS

(length, size, type...). Estas se utilizan para validación y sugerencia de corrección. Como sabemos el lexer produce una lista de tokens anteriormente vista en la etapa anterior, el parser recibe esta lista y la procesa secuencialmente, avanzando posición por posición sobre cada uno de los tokens entregados, si hay errores léxicos, el lexer los reporta inicialmente, pero el parser asume tokens únicamente validos y se enfoca en revisar errores sintácticos sobre ellos (alguna secuencia incorrecta, o estructura mal redactada). Lo hace basándose en si el token actual coincide con el tipo de token que fue declarado y con el valor si es necesario **(es decir hay tokens donde mi valor realmente no importan al parser como el valor de un IDENTIFIER, pero si es importante conocer el valor de un token marcado como KEYWORD para detectar precisamente cuál será su estructura buscada)**, si no es así entonces registra un error (esperado vs encontrado), si el token es correcto entonces avanza.

3.3 Declaraciones BNF de HOOP

<code><programa> ::= (<declaracion> <clase> <funcion> <statement>)* <EOF></code>
<code><declaracion> ::= "data" <IDENTIFIER> "set" <expresion> ";"</code>
<code><funcion> ::= "action" <IDENTIFIER> "(" <parametros>? ")" "{" <statements> "}"</code>
<code><funcion_global> ::= "action" <IDENTIFIER> "(" <parametros>? ")" "{" <statements> "}"</code>
<code><parametros> ::= <parametro> ("," <parametro>)*</code>
<code><parametros> ::= <TIPO> <IDENTIFIER> ("," <TIPO> <IDENTIFIER>)*</code>
<code><clase> ::= "mold" <IDENTIFIER> "{" <clase_cuerpo> "}"</code>
<code><clase_cuerpo> ::= (<atributo> <metodo>)*</code>
<code><statements> ::= (<statement>)*</code> <code><statement> ::= <if_statement></code> <code> <cycle_statement></code> <code> <simple_statement></code> <code> <declaracion></code>
<code><if_statement> ::= "when" <condicion> "{" <statements> "}" ["otherwise" "{" <statements> "}"]</code>

<cycle_statement> ::= "cycle" <IDENTIFIER> ["from" <expresion>] ["to" <expresion>] "{" <statements> "}"

<simple_statement> ::= <expresion> ";"

<expresion> ::= <literal>

<literal> ::= <STRING> | <NUMBER> | <BOOLEAN>

<operacion> ::= <IDENTIFIER> <OPERADOR> <IDENTIFIER>

<condicion> ::= <expresion> <OPERADOR_COMPARACION> <expresion>

<OPERADOR> ::= "plus" | "minus" | "times" | "divide" | "mod"

<OPERADOR_COMPARACION> ::= "equals" | "notequals" | "greater" | "less" | "greatereq" | "lesseq"

<TIPO> ::= "logic" | "whole" | "fract" | "text" | "char" | "grid" | "chain"

<KEYWORD> ::= "mold" | "data" | "action" | "when" | "otherwise" | "cycle" | "from" | "to" | "set"